AIEP
de la Universidad Andrés Bello

Automatización del inventario con SQL Server

Estudiante: Blas Pinto

Módulo: Taller de Bases de Datos

Docente: Marcelo Trigo

Semana: 9

Fecha de elaboración: 9 de octubre de 2026

Tabla de contenidos

Introducción.....3

Desarrollo.....4

Conclusión.....13

Introducción

En este informe presento una solución para registrar altas y bajas de productos y conservar un historial de los movimientos del inventario. El caso corresponde a una empresa que vende productos electrónicos y necesita automatizar estas operaciones. La propuesta utiliza procedimientos almacenados para gestionar los productos y triggers para registrar los cambios en una tabla de auditoría.

La implementación se probó en SQL Server 2022 LocalDB con productos ficticios. Los resultados mostraron que las altas y bajas quedaron registradas, las entradas inválidas fueron rechazadas y las actualizaciones sin cambios no generaron movimientos adicionales. En la ejecución ampliada, el estado final fue de cero productos y diez registros de auditoría: cuatro INS, cuatro DEL y dos UPD.

El objetivo es implementar las tablas Productos y AuditoriaInventario, controlar las operaciones mediante parámetros y manejo de excepciones, y verificar los resultados con consultas y evidencias. Además de los pasos del caso, se incorporó una actualización condicional y una demostración de cursor para abordar los aspectos mencionados en la rúbrica de la semana 9 [1].

Las capturas muestran un visor local de registros obtenidos mediante .NET SqlClient. Sus consultas, mensajes y resultados proceden de ejecuciones reales; no corresponden a la interfaz de SQL Server Management Studio. La fecha de las pruebas es el 8 de octubre de 2026. El informe se elaboró el 9 de octubre de 2026.

Desarrollo

Entorno y alcance de las pruebas

El motor utilizado fue Microsoft SQL Server 2022 Express LocalDB, versión 16.0.1000.6, en Windows. La conexión se realizó a la instancia (localdb)\MSSQLLocalDB con autenticación de Windows. La serie de evidencias de este informe se ejecutó en una base separada denominada PRO202_Evidencias_20261008_163918, creada exclusivamente para la actividad.

Las pruebas siguieron una secuencia común: consultar el estado anterior, ejecutar la operación y revisar el estado posterior y los mensajes. El archivo Registro_completo.txt conserva esa secuencia. Los identificadores F01 a F35 permiten localizar los extractos y sus capturas. El anexo de evidencias incluye una selección de los resultados principales y conserva esos identificadores de origen.

Diseño de las tablas

Productos conserva los datos de cada artículo. Su identificador se genera con IDENTITY y funciona como clave primaria. AuditoriaInventario guarda el identificador del producto afectado, el tipo de acción y la fecha generada por GETDATE(). El esquema base proviene del caso de la actividad [1, p. 2].

Tabla	Campo	Tipo y función
Productos	IdProducto	INT, PRIMARY KEY e IDENTITY
Productos	Nombre	NVARCHAR(100), nombre del producto
Productos	Stock	INT, cantidad disponible
Productos	Precio	DECIMAL(10,2), valor del producto
AuditoriaInventario	IdAuditoria	INT, PRIMARY KEY e IDENTITY
AuditoriaInventario	IdProducto	INT, producto asociado al movimiento
AuditoriaInventario	Accion	NVARCHAR(10), INS, DEL o UPD
AuditoriaInventario	Fecha	DATETIME, DEFAULT GETDATE()

Como decisión de diseño, se añadieron restricciones para impedir nombres nulos o vacíos, stock negativo o nulo y precio negativo o nulo. Se permiten el stock cero y el precio cero; esta decisión se verificó con una prueba que después se revirtió mediante ROLLBACK. El enunciado no define estas reglas numéricas, por lo que se presentan como validaciones propuestas para este ejercicio.

No se creó una clave foránea desde AuditoriaInventario hacia Productos. Si el producto se elimina, el historial debe conservar su identificador. Esta solución permite registrar bajas, aunque no guarda una copia del nombre o precio histórico. Para un sistema real, sería conveniente ampliar los datos de auditoría según las necesidades del negocio.

Procedimientos almacenados

Alta de productos

sp_AltaProducto recibe @Nombre, @Stock y @Precio. Primero valida los parámetros y luego inserta el producto dentro de BEGIN TRY. Si se produce un error, BEGIN CATCH muestra ERROR_MESSAGE() y THROW lo propaga al programa que ejecutó la consulta. Así, el fallo queda registrado y no se informa una inserción exitosa cuando los datos fueron rechazados [3].

La prueba válida utilizó un Teclado de prueba con stock 10 y precio 19990. Antes de la operación, ambas tablas estaban vacías. Después se creó el producto con ID 1 y un registro INS con ese mismo identificador. El mensaje fue «Producto insertado correctamente.». La Figura 1 permite comparar los estados anterior y posterior.

Baja de productos

sp_BajaProducto recibe @IdProducto y exige que sea positivo. El procedimiento ejecuta DELETE y revisa @@ROWCOUNT inmediatamente después. Si no se eliminó ninguna fila, muestra «Producto no existe.»; si se eliminó una fila, informa que el producto fue eliminado. Esta comprobación mantiene el comportamiento solicitado por el ejemplo con IF EXISTS, sin separar la comprobación de la operación.

Con el ID 1 existente, el producto desapareció de Productos y se agregó una acción DEL en AuditoriaInventario. El registro INS anterior se mantuvo. Cuando se solicitó eliminar el ID 2147483647, no hubo cambios y se

mostró el mensaje de inexistencia. Las Figuras 2 y 3 documentan ambas situaciones.

Triggers y registro de auditoría

Los triggers AFTER INSERT y AFTER DELETE toman las filas afectadas desde inserted y deleted, respectivamente, y las registran mediante INSERT ... SELECT. Esta lógica trabaja con todas las filas de una operación, lo que se comprobó al insertar y eliminar dos productos en una sola sentencia. No se utiliza un cursor dentro de los triggers.

Objeto	Tipo	Función
trg_Auditoria_AltaProducto	AFTER INSERT	Registrar INS por cada fila insertada
trg_Auditoria_BajaProducto	AFTER DELETE	Registrar DEL por cada fila eliminada
trg_Auditoria_ActualizarProducto	AFTER UPDATE	Registrar UPD solo cuando cambia un valor
trg_Validar_Actualizacion	INSTEAD OF UPDATE	Rechazar valores inválidos y aplicar cambios válidos

El trigger de actualización compara Nombre, Stock y Precio entre inserted y deleted. Si no cambia ninguno de esos valores, no registra UPD. En la prueba, los stocks de los productos ID 2 y 3 pasaron de 3 y 5 a 4 y 6, y se crearon dos registros UPD. Al ejecutar después SET Stock = Stock, el número de registros se mantuvo. Las Figuras 8 y 9 muestran estos resultados.

La rúbrica menciona BEFORE, mientras que SQL Server ofrece AFTER e INSTEAD OF para este tipo de triggers [2]. Por esa razón, se implementó INSTEAD OF UPDATE: revisa los valores y, si son válidos, realiza la actualización. Este tipo reemplaza la operación original; no es un trigger BEFORE. La diferencia se deja explícita para que pueda evaluarse considerando el motor utilizado.

Cuando se intentó asignar stock -1, el trigger generó el error 50005 y mantuvo los valores anteriores. También se rechazaron nombre nulo o vacío, stock nulo y precio nulo o negativo. La consulta sp_helptrigger identificó los cuatro triggers y sus tipos; su salida se presenta en la Figura 4.

Manejo de excepciones y cursor

La inexistencia de un producto es una condición controlada del procedimiento de baja y se comunica mediante un mensaje. En cambio, un identificador negativo o un stock negativo generan errores de validación. Estas situaciones activaron CATCH en los procedimientos y quedaron registradas como errores esperados. Después de ambos rechazos, no se añadieron productos ni movimientos de auditoría. La Figura 5 contiene las consultas, los mensajes y el estado posterior.

Para la parte de cursores se creó sp_ConsultarInventarioCursor. Utiliza un cursor LOCAL FAST_FORWARD sobre los productos ordenados por ID. El procedimiento abre el cursor, obtiene cada fila con FETCH NEXT y muestra el identificador, nombre y stock. Al terminar, lo cierra y libera con CLOSE

y DEALLOCATE. Se eligió este cursor para una demostración de recorrido de lectura hacia adelante [4].

El cursor mostró los productos ID 2 y 3 con stocks 3 y 5. También se ejecutó cuando Productos estaba vacío; no emitió líneas de productos ni errores. Las Figuras 7 y 11 documentan ambos casos. El código incluye limpieza de recursos en CATCH, pero no se provocó un error dentro del recorrido. Para listar un inventario en una aplicación, un SELECT sería suficiente; el cursor se incluye aquí con una finalidad académica.

Verificación de funcionamiento

La siguiente matriz reúne las pruebas ejecutadas, sus parámetros y los resultados observados. Los identificadores Fxx remiten al registro de evidencias; las referencias a figuras corresponden a las capturas incluidas al final del informe. Los datos utilizados son ficticios y no representan un inventario productivo.

Prueba	Parámetro u operación	Resultado observado	Evidencia
Alta válida	N'Teclado de prueba', 10, 19990	Producto ID 1 y auditoría INS	F07 / Figura 1
Baja válida	ID 1 existente	Producto eliminado y auditoría DEL	F08 / Figura 2
Baja inexistente	ID 2147483647	Mensaje de inexistencia; sin cambios	F09 / Figura 3
Inspección	sp_helptrigger Productos	Cuatro triggers con sus tipos	F10 / Figura 4
Definición alta	sp_helptext sp_AltaProducto	Definición real recuperada	F11

Prueba	Parámetro u operación	Resultado observado	Evidencia
Definición baja	sp_helptext sp_BajaProducto	Definición real recuperada	F12
Stock negativo	Alta con stock -1	Error 50002 y mensaje de CATCH	F13 / Figura 5
ID negativo	Baja con ID -1	Error 50004 y mensaje de CATCH	F13 / Figura 5
Inserción múltiple	Monitor stock 3; Mouse stock 5	Dos productos y dos INS adicionales	F14 / Figura 6
Cursor con datos	Dos productos	Dos mensajes de productos	F15 / Figura 7
UPDATE válido	Stock = Stock + 1	Stocks 4 y 6; dos registros UPD	F16 / Figura 8
UPDATE sin cambios	Stock = Stock	No agrega registros UPD	F17 / Figura 9
UPDATE inválido	Stock = -1	Error 50005; datos conservados	F18 / Figura 10
Eliminación múltiple	DELETE de dos productos	Dos DEL; tabla Productos vacía	F19
Cursor vacío	Productos con cero filas	Sin mensajes de productos ni error	F20 / Figura 11
Nombre nulo	Alta con Nombre = NULL	Error 50001; sin inserción	F21
Nombre vacío	Alta con Nombre vacío	Error 50001; sin inserción	F22
Nombre con espacios	Alta con tres espacios	Error 50001; sin inserción	F23
Stock nulo	Alta con Stock = NULL	Error 50002; sin inserción	F24

Prueba	Parámetro u operación	Resultado observado	Evidencia
Precio nulo	Alta con Precio = NULL	Error 50003; sin inserción	F25
Precio negativo	Alta con Precio = -10	Error 50003; sin inserción	F26
ID nulo	Baja con ID = NULL	Error 50004; sin eliminación	F27
ID cero	Baja con ID = 0	Error 50004; sin eliminación	F28
Límites válidos	Stock 0 y Precio 0	Inserción aceptada y luego ROLLBACK	F29
UPDATE nombre nulo	Nombre = NULL	Error 50005; fila conservada	F30
UPDATE nombre vacío	Nombre vacío	Error 50005; fila conservada	F31
UPDATE stock nulo	Stock = NULL	Error 50005; fila conservada	F32
UPDATE precio nulo	Precio = NULL	Error 50005; fila conservada	F33
UPDATE precio negativo	Precio = -1	Error 50005; fila conservada	F34
Comprobación final	Recuentos con validación automática	0 productos, 4 INS, 4 DEL y 2 UPD	F35 / Figura 12

Los casos con error esperado verificaron que el motor devolviera un código de validación entre 50001 y 50005. El ejecutor se detiene si no aparece el error previsto o si la causa no corresponde a un error SQL de validación. Las consultas posteriores permiten revisar que los rechazos no agregaron filas ni auditorías indebidas.

El producto de la prueba con stock y precio cero se insertó dentro de una transacción y luego se revirtió. Por eso, su registro INS no forma parte del recuento final. En las validaciones adicionales de UPDATE se utilizó otro producto confirmado previamente; al terminar, se eliminó y quedaron registradas su alta y su baja. Esto explica que la serie ampliada finalice con cuatro INS y cuatro DEL, además de los dos UPD.

La verificación final comprobó los recuentos y mostró «Verificaciones superadas.». La Figura 12 incluye la consulta y la salida. Las pruebas cubren los casos del ejercicio y las validaciones descritas; no se realizaron pruebas de concurrencia, fallos de conexión ni todos los errores posibles del motor.

Claridad del código y decisiones de implementación

Los objetos se organizan en el esquema dbo y sus nombres describen la operación que realizan. Los comentarios explican la conservación de auditoría, la comparación de cambios y el uso académico del cursor. SET NOCOUNT ON evita mensajes de filas afectadas dentro de los objetos. Las validaciones también se respaldan con restricciones CHECK en Productos, de modo que no dependen únicamente de llamar a los procedimientos.

La auditoría de múltiples filas se resuelve con consultas por conjuntos. El cursor se mantiene en un procedimiento separado y no interviene en el registro de altas, bajas o actualizaciones. El anexo de código distingue la implementación del caso de la ampliación propuesta a partir de la rúbrica.

Conclusión

La solución permitió insertar y eliminar productos mediante procedimientos almacenados y registrar sus movimientos de forma automática. Las consultas mostraron que el historial se conserva después de una baja y que una operación con varias filas genera una auditoría por cada producto afectado. La ejecución ampliada terminó con los recuentos esperados y sin productos pendientes en la base de pruebas.

La primera reflexión que deja este trabajo es que el manejo de errores debe comprobarse con datos inválidos, además de probar operaciones correctas. Un mensaje de inserción exitosa por sí solo no permite saber si las validaciones funcionan. Comparar los estados antes y después ayudó a distinguir una baja inexistente, un rechazo por parámetros y un cambio efectivamente registrado.

La segunda reflexión es que una solución debe ajustarse al motor elegido. La mención BEFORE de la pauta requirió explicar el uso de INSTEAD OF en SQL Server. También fue necesario distinguir la demostración de un cursor de la forma más sencilla de consultar los productos. Para proyectar esta propuesta a un sistema real, ampliaría la información histórica de auditoría y evaluaría permisos y operaciones concurrentes, aspectos que quedan fuera de estas pruebas.

Referencias

[1] AIEP. Orientaciones y pauta de evaluación. Actividad sumativa, semana 9. Taller de Bases de Datos, unidad 2. Documento de la actividad, 8 páginas. Requisitos: pp. 1–5; rúbrica: pp. 6–8.

[2] Microsoft Learn. CREATE TRIGGER (Transact-SQL). Consultado el 9 de octubre de 2026. https://learn.microsoft.com/en-us/sql/t-sql/statements/create-trigger-transact-sql

[3] Microsoft Learn. TRY...CATCH (Transact-SQL). Consultado el 9 de octubre de 2026. https://learn.microsoft.com/en-us/sql/t-sql/language-elements/try-catch-transact-sql

[4] Microsoft Learn. DECLARE CURSOR (Transact-SQL). Consultado el 9 de octubre de 2026. https://learn.microsoft.com/en-us/sql/t-sql/language-elements/declare-cursor-transact-sql

Asistencia utilizada

Se utilizó Codex como apoyo para organizar los requisitos, preparar el código, ejecutar las pruebas locales, capturar el visor de resultados y redactar el informe. Los resultados provienen de SQL Server y los productos utilizados son ficticios de prueba. La autoría académica del informe corresponde a Blas Pinto.

Anexo de código SQL

Los dos scripts se ejecutan en el orden indicado sobre una base nueva y vacía. No deben repetirse sobre una base con estos objetos ya creados. GO separa los lotes de definición. A continuación se incluye el código utilizado en las pruebas, sin reemplazarlo por una versión distinta.

Implementación del caso

```
-- Actividad semana 9. Ejecutar en una base exclusiva de pruebas.
-- Las validaciones son decisiones de diseño, no reglas textuales del PDF.
CREATE TABLE dbo.Productos (
 IdProducto INT PRIMARY KEY IDENTITY,
 Nombre NVARCHAR(100), Stock INT, Precio DECIMAL(10,2),
 CONSTRAINT CK_Productos_Nombre CHECK (Nombre IS NOT NULL AND
LEN(LTRIM(RTRIM(Nombre))) > 0),
 CONSTRAINT CK_Productos_Stock CHECK (Stock IS NOT NULL AND Stock >= 0),
 CONSTRAINT CK_Productos_Precio CHECK (Precio IS NOT NULL AND Precio >= 0)
);
CREATE TABLE dbo.AuditoriaInventario (
 IdAuditoria INT PRIMARY KEY IDENTITY,
 IdProducto INT, Accion NVARCHAR(10), Fecha DATETIME DEFAULT GETDATE()
);
-- Sin FK a Productos: la auditoría debe conservar las bajas.
GO
CREATE PROCEDURE dbo.sp_AltaProducto
 @Nombre NVARCHAR(100), @Stock INT, @Precio DECIMAL(10,2)
AS
BEGIN
 SET NOCOUNT ON;
 BEGIN TRY
  IF @Nombre IS NULL OR LEN(LTRIM(RTRIM(@Nombre)))=0
   THROW 50001, 'El nombre no puede estar vacío.', 1;
  IF @Stock IS NULL OR @Stock < 0 THROW 50002, 'El stock debe ser no negativo.', 1;
  IF @Precio IS NULL OR @Precio < 0 THROW 50003, 'El precio debe ser no negativo.', 1;
  INSERT dbo.Productos(Nombre,Stock,Precio) VALUES(@Nombre,@Stock,@Precio);
  PRINT 'Producto insertado correctamente.';
 END TRY
 BEGIN CATCH
  PRINT 'Error al insertar el producto: ' + ERROR_MESSAGE();
  THROW;
 END CATCH
END;
GO
```

```
CREATE TRIGGER dbo.trg_Auditoria_AltaProducto ON dbo.Productos AFTER INSERT
AS
BEGIN
 SET NOCOUNT ON;
 INSERT dbo.AuditoriaInventario(IdProducto,Accion)
 SELECT IdProducto,'INS' FROM inserted;
END;
GO
CREATE TRIGGER dbo.trg_Auditoria_BajaProducto ON dbo.Productos AFTER DELETE
AS
BEGIN
 SET NOCOUNT ON;
 INSERT dbo.AuditoriaInventario(IdProducto,Accion)
 SELECT IdProducto,'DEL' FROM deleted;
END;
GO
CREATE PROCEDURE dbo.sp_BajaProducto @IdProducto INT
AS
BEGIN
 SET NOCOUNT ON;
 BEGIN TRY
  IF @IdProducto IS NULL OR @IdProducto <= 0
   THROW 50004, 'El identificador debe ser positivo.', 1;
  -- DELETE y @@ROWCOUNT evitan separar la comprobación de la operación.
  DELETE dbo.Productos WHERE IdProducto=@IdProducto;
  IF @@ROWCOUNT=0 PRINT 'Producto no existe.';
  ELSE PRINT 'Producto eliminado correctamente.';
 END TRY
 BEGIN CATCH
  PRINT 'Error al eliminar el producto: ' + ERROR_MESSAGE();
  THROW;
 END CATCH
END;
GO
```

Ampliación para actualización y cursor

```
-- Ampliación propuesta para cubrir menciones de la rúbrica.
-- INSTEAD OF reemplaza la operación; no es un trigger BEFORE.
CREATE TRIGGER dbo.trg_Validar_Actualizacion ON dbo.Productos INSTEAD OF UPDATE
AS
BEGIN
 SET NOCOUNT ON;
 IF EXISTS(SELECT 1 FROM inserted WHERE Nombre IS NULL OR LEN(LTRIM(RTRIM(Nombre)))=0
  OR Stock IS NULL OR Stock<0 OR Precio IS NULL OR Precio<0)
  THROW 50005, 'Actualización rechazada: nombre, stock o precio inválidos.', 1;
 UPDATE p SET Nombre=i.Nombre, Stock=i.Stock, Precio=i.Precio
 FROM dbo.Productos p JOIN inserted i ON p.IdProducto=i.IdProducto;
END;
GO
CREATE TRIGGER dbo.trg_Auditoria_ActualizarProducto ON dbo.Productos AFTER UPDATE
AS
```

```
BEGIN
 SET NOCOUNT ON;
 -- Condicional: no registrar una actualización sin cambios efectivos.
 INSERT dbo.AuditoriaInventario(IdProducto,Accion)
 SELECT i.IdProducto,'UPD' FROM inserted i JOIN deleted d ON i.IdProducto=d.IdProducto
 WHERE i.Nombre<>d.Nombre OR i.Stock<>d.Stock OR i.Precio<>d.Precio;
END;
GO
CREATE PROCEDURE dbo.sp_ConsultarInventarioCursor
AS
BEGIN
 SET NOCOUNT ON;
 DECLARE @Id INT,@Nombre NVARCHAR(100),@Stock INT;
 -- Demostración académica de recorrido. Para listar inventario se prefiere SELECT.
 DECLARE inventario CURSOR LOCAL FAST_FORWARD FOR
 SELECT IdProducto,Nombre,Stock FROM dbo.Productos ORDER BY IdProducto;
 BEGIN TRY
  OPEN inventario;
  FETCH NEXT FROM inventario INTO @Id,@Nombre,@Stock;
  WHILE @@FETCH_STATUS=0
  BEGIN
   PRINT CONCAT('ID=',@Id,'; Nombre=',@Nombre,'; Stock=',@Stock);
   FETCH NEXT FROM inventario INTO @Id,@Nombre,@Stock;
  END;
  CLOSE inventario;
  DEALLOCATE inventario;
 END TRY
 BEGIN CATCH
  IF CURSOR_STATUS('local','inventario')>=0 CLOSE inventario;
  IF CURSOR_STATUS('local','inventario')>=-1 DEALLOCATE inventario;
  THROW;
 END CATCH
END;
GO
```

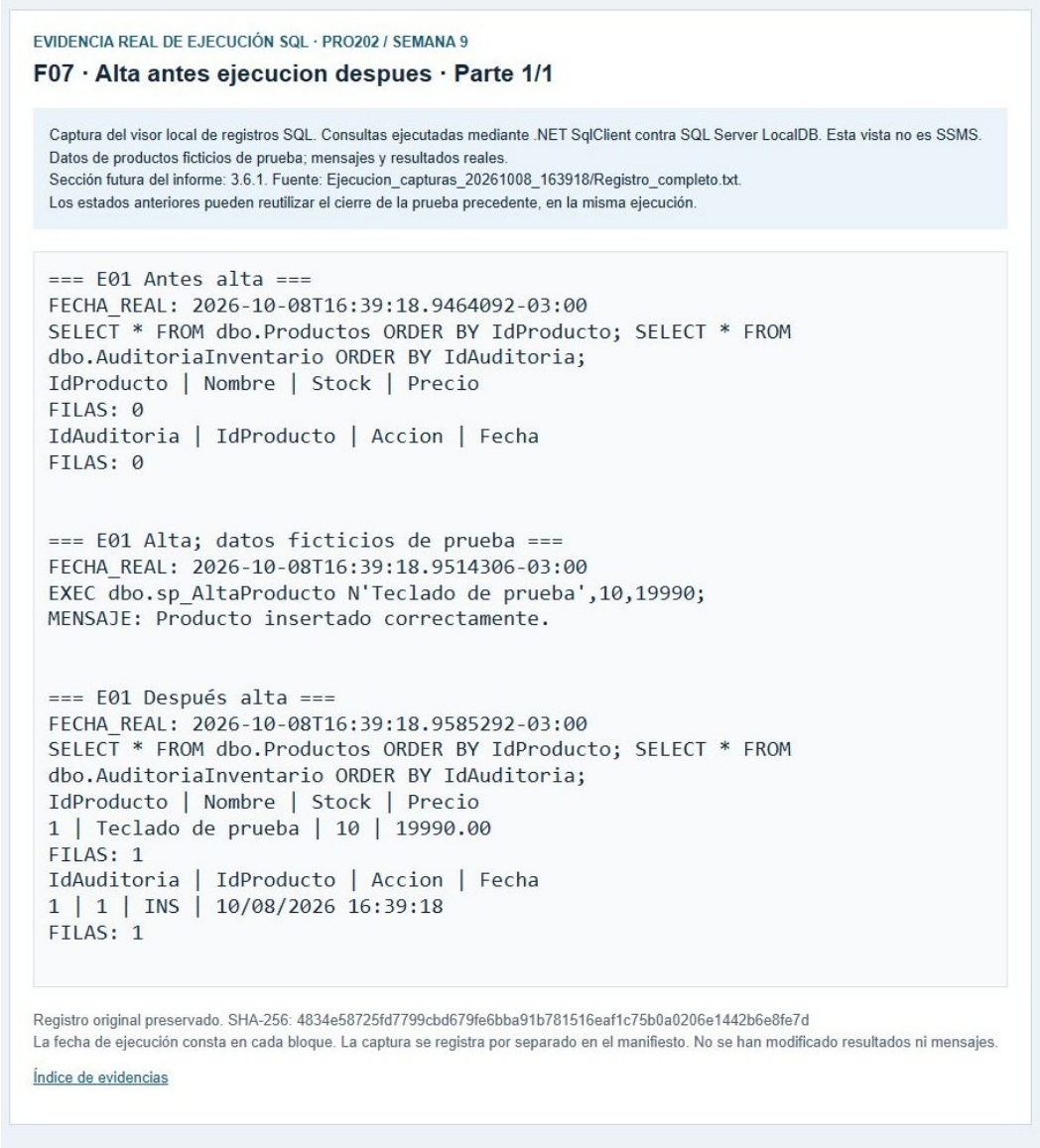
Anexo de evidencias

Las siguientes capturas muestran las pruebas principales. Cada figura conserva el ID del registro Fxx y, cuando corresponde, se divide en partes. Son capturas de un visor local de resultados de SQL Server obtenidas el 9 de octubre de 2026 a partir de la ejecución del 8 de octubre. Se eliminaron únicamente los márgenes vacíos de las copias para lectura; las capturas completas se conservaron.

El registro de evidencias contiene 35 conjuntos y 57 capturas paginadas de la serie inicial. La matriz de verificación documenta también las pruebas complementarias que no se repiten como imágenes en este anexo. El repositorio del proyecto conserva el registro completo, los archivos SQL y todas las capturas.

Figura 1 Alta de producto y auditoría INS

Tablas vacías antes de la operación; producto ID 1 y registro INS después.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F07 · Alta antes ejecucion despues · Parte 1/1

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.1. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E01 Antes alta ===
FECHA_REAL: 2026-10-08T16:39:18.9464092-03:00
SELECT * FROM dbo.Productos ORDER BY IdProducto; SELECT * FROM
dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
FILAS: 0
IdAuditoria | IdProducto | Accion | Fecha
FILAS: 0


=== E01 Alta; datos ficticios de prueba ===
FECHA_REAL: 2026-10-08T16:39:18.9514306-03:00
EXEC dbo.sp_AltaProducto N'Teclado de prueba',10,19990;
MENSAJE: Producto insertado correctamente.


=== E01 Después alta ===
FECHA_REAL: 2026-10-08T16:39:18.9585292-03:00
SELECT * FROM dbo.Productos ORDER BY IdProducto; SELECT * FROM
dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
1 | Teclado de prueba | 10 | 19990.00
FILAS: 1
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
FILAS: 1
```

Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 1. Alta de producto y auditoría INS. Evidencia F07, parte 1/1. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 2 Baja de producto y auditoría DEL

El producto ID 1 desaparece y su historial INS y DEL permanece.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F08 · Baja antes ejecucion despues · Parte 1/1

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.2. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E01 Después alta ===
FECHA_REAL: 2026-10-08T16:39:18.9585292-03:00
SELECT * FROM dbo.Productos ORDER BY IdProducto; SELECT * FROM
dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
1 | Teclado de prueba | 10 | 19990.00
FILAS: 1
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
FILAS: 1


=== E02 Baja ID existente ===
FECHA_REAL: 2026-10-08T16:39:18.9631713-03:00
DECLARE @Id INT=(SELECT MIN(IdProducto) FROM dbo.Productos); EXEC
dbo.sp_BajaProducto @Id;
MENSAJE: Producto eliminado correctamente.


=== E02 Después baja ===
FECHA_REAL: 2026-10-08T16:39:18.9681036-03:00
SELECT * FROM dbo.Productos ORDER BY IdProducto; SELECT * FROM
dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
FILAS: 0
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
FILAS: 2
```

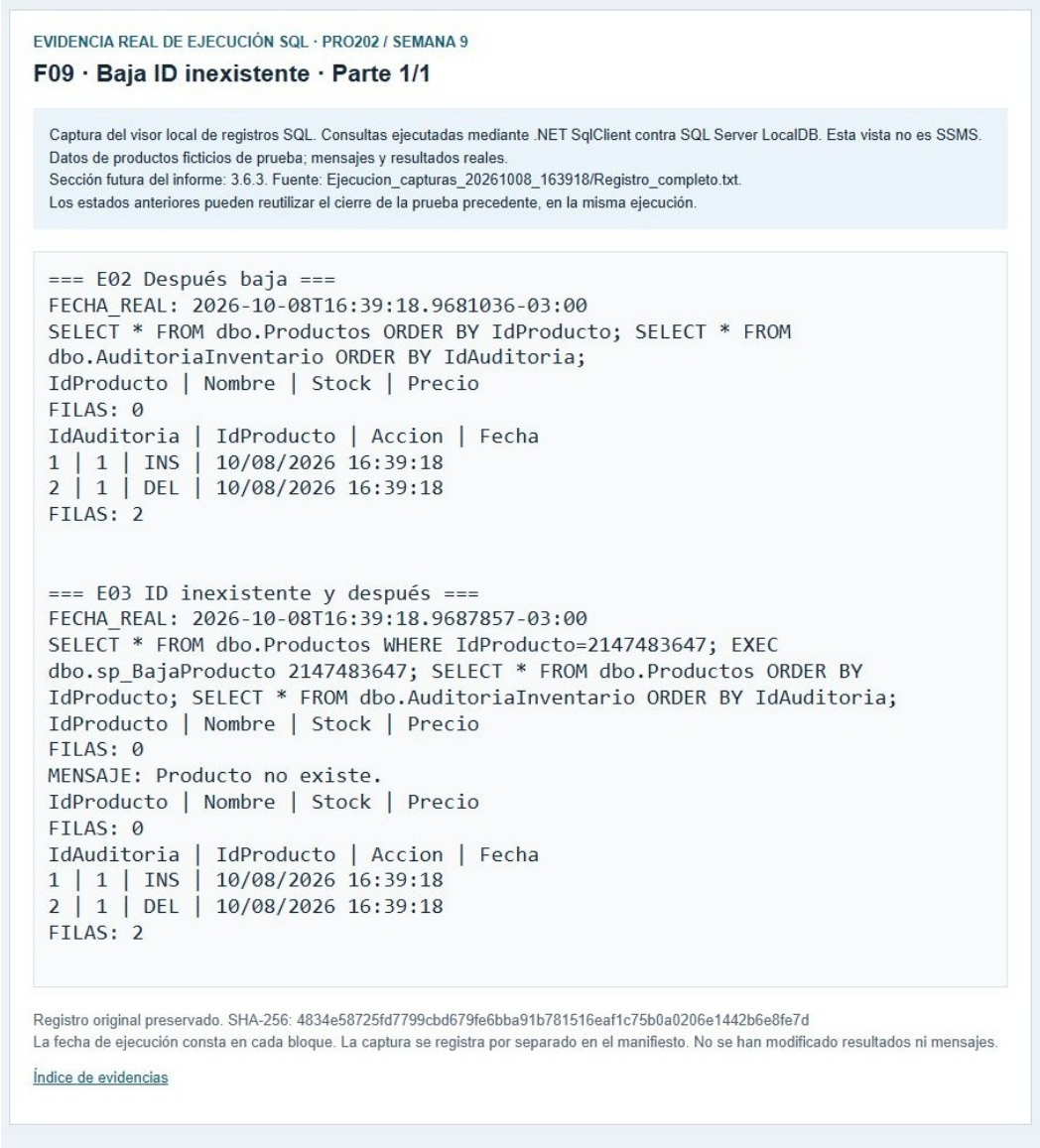
Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 2. Baja de producto y auditoría DEL. Evidencia F08, parte 1/1. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 3 Baja con identificador inexistente

El ID consultado no existe; el mensaje y la auditoría confirman la ausencia de cambios.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F09 · Baja ID inexistente · Parte 1/1

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.3. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E02 Después baja ===
FECHA_REAL: 2026-10-08T16:39:18.9681036-03:00
SELECT * FROM dbo.Productos ORDER BY IdProducto; SELECT * FROM
dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
FILAS: 0
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
FILAS: 2


=== E03 ID inexistente y después ===
FECHA_REAL: 2026-10-08T16:39:18.9687857-03:00
SELECT * FROM dbo.Productos WHERE IdProducto=2147483647; EXEC
dbo.sp_BajaProducto 2147483647; SELECT * FROM dbo.Productos ORDER BY
IdProducto; SELECT * FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
FILAS: 0
MENSAJE: Producto no existe.
IdProducto | Nombre | Stock | Precio
FILAS: 0
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
FILAS: 2
```

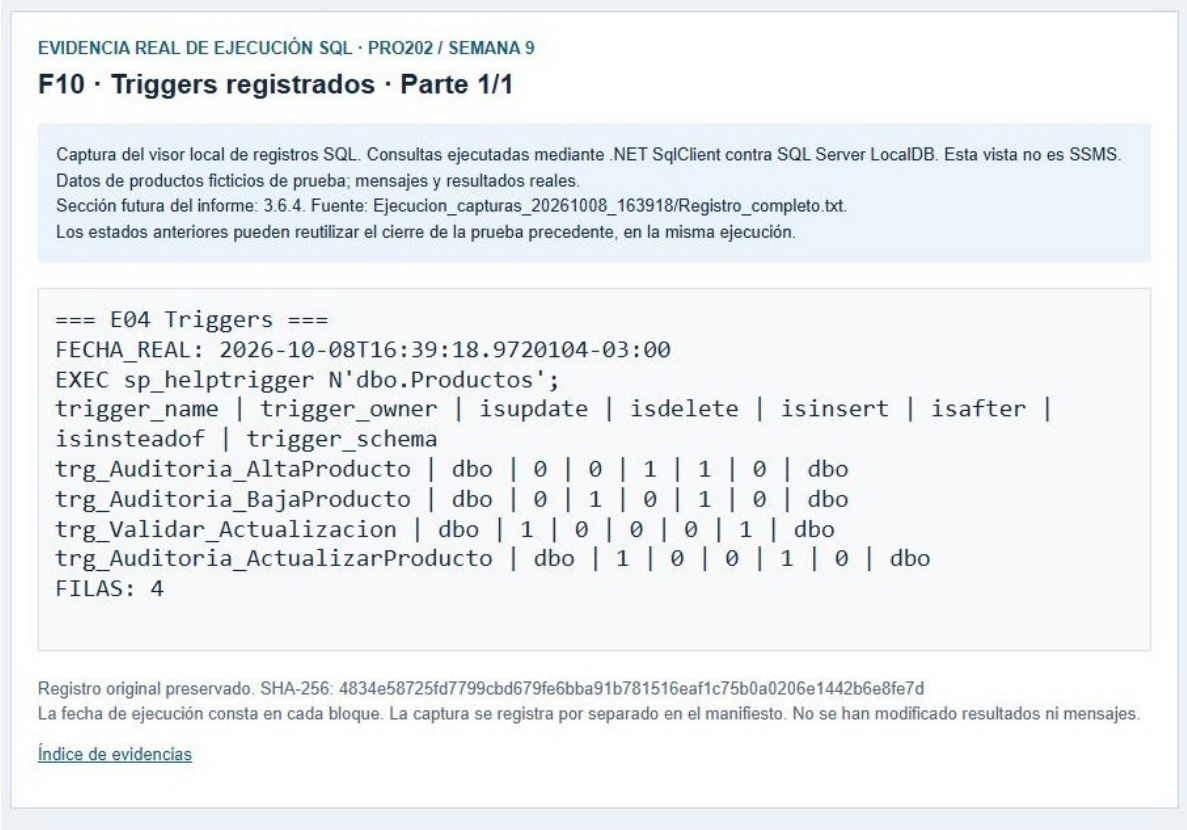
Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 3. Baja con identificador inexistente. Evidencia F09, parte 1/1. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 4 Verificación de triggers

sp_helptrigger informa los cuatro triggers y sus eventos.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F10 · Triggers registrados · Parte 1/1

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.4. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E04 Triggers ===
FECHA_REAL: 2026-10-08T16:39:18.9720104-03:00
EXEC sp_helptrigger N'dbo.Productos';
trigger_name | trigger_owner | isupdate | isdelete | isinsert | isafter |
isinsteadof | trigger_schema
trg_Auditoria_AltaProducto | dbo | 0 | 0 | 1 | 1 | 0 | dbo
trg_Auditoria_BajaProducto | dbo | 0 | 1 | 0 | 1 | 0 | dbo
trg_Validar_Actualizacion | dbo | 1 | 0 | 0 | 0 | 1 | dbo
trg_Auditoria_ActualizarProducto | dbo | 1 | 0 | 0 | 1 | 0 | dbo
FILAS: 4
```

Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 4. Verificación de triggers. Evidencia F10, parte 1/1. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 5 Excepciones de alta y baja

Stock negativo e ID negativo rechazados; se muestran los mensajes y el estado posterior.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F13 · Excepciones alta y baja · Parte 1/2

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.5. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E03 ID inexistente y después ===
FECHA_REAL: 2026-10-08T16:39:18.9687857-03:00
SELECT * FROM dbo.Productos WHERE IdProducto=2147483647; EXEC
dbo.sp_BajaProducto 2147483647; SELECT * FROM dbo.Productos ORDER BY
IdProducto; SELECT * FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
FILAS: 0
MENSAJE: Producto no existe.
IdProducto | Nombre | Stock | Precio
FILAS: 0
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
FILAS: 2


=== E07 CATCH alta ===
FECHA_REAL: 2026-10-08T16:39:19.1068900-03:00
EXEC dbo.sp_AltaProducto N'Inválido',-1,10;
MENSAJE: Error al insertar el producto: El stock debe ser no negativo.
ERROR: Exception calling "ExecuteReader" with "0" argument(s): "El stock debe
ser no negativo."
NUMERO_ERROR_SQL: 50002


=== E07 CATCH baja ===
FECHA_REAL: 2026-10-08T16:39:19.1224721-03:00
EXEC dbo.sp_BajaProducto -1;
MENSAJE: Error al eliminar el producto: El identificador debe ser positivo.
ERROR: Exception calling "ExecuteReader" with "0" argument(s): "El
identificador debe ser positivo."
```

Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 5. Excepciones de alta y baja. Evidencia F13, parte 1/2. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 5 Excepciones de alta y baja

Stock negativo e ID negativo rechazados; se muestran los mensajes y el estado posterior.

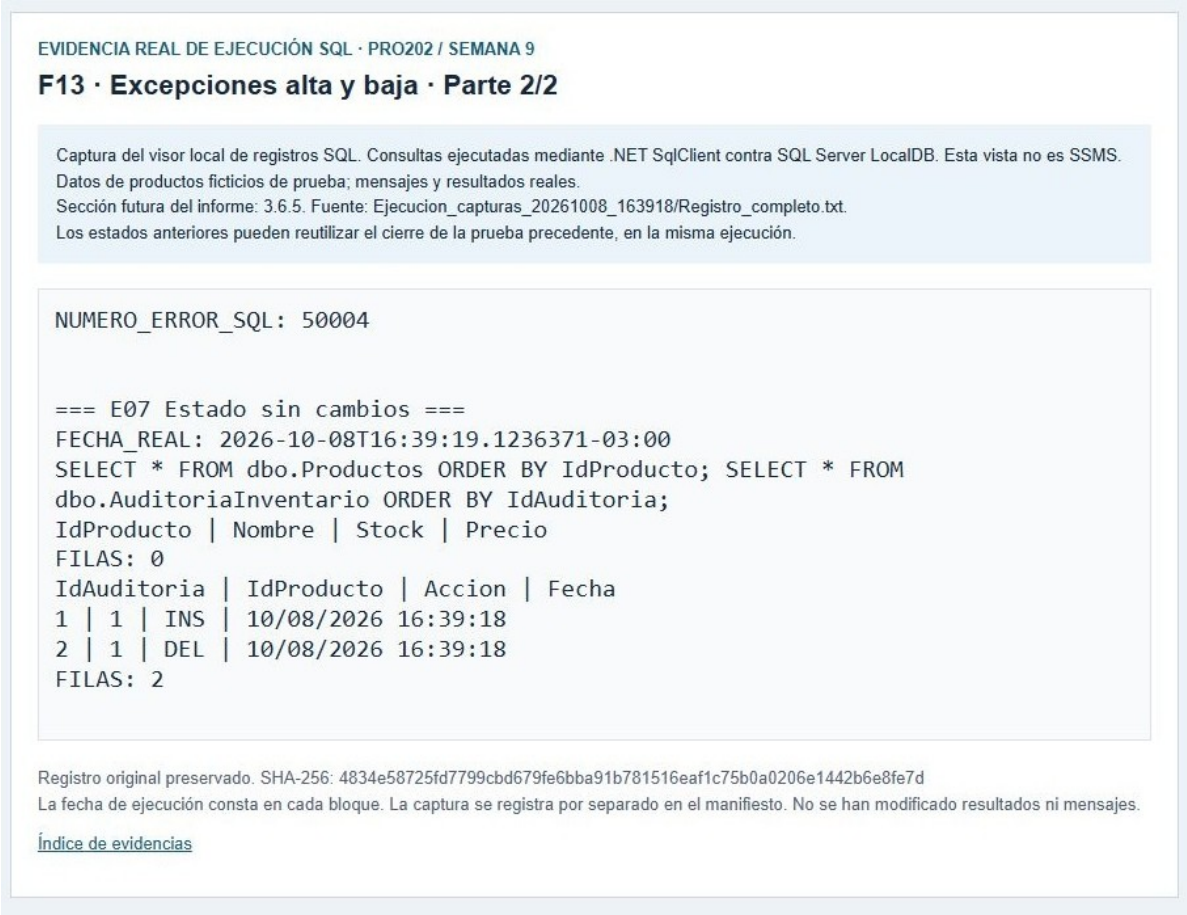


Figura 5. Excepciones de alta y baja. Evidencia F13, parte 2/2. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 6 Inserción de varias filas

Dos productos nuevos generan dos registros INS adicionales.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F14 · Insercion multiple · Parte 1/1

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.6. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E07 Estado sin cambios ===
FECHA_REAL: 2026-10-08T16:39:19.1236371-03:00
SELECT * FROM dbo.Productos ORDER BY IdProducto; SELECT * FROM
dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
FILAS: 0
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
FILAS: 2


=== E08 Alta múltiple ===
FECHA_REAL: 2026-10-08T16:39:19.1243335-03:00
INSERT dbo.Productos(Nombre,Stock,Precio) VALUES(N'Monitor prueba',3,100),
(N'Mouse prueba',5,20); SELECT * FROM dbo.Productos ORDER BY IdProducto;
SELECT * FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
2 | Monitor prueba | 3 | 100.00
3 | Mouse prueba | 5 | 20.00
FILAS: 2
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
3 | 3 | INS | 10/08/2026 16:39:19
4 | 2 | INS | 10/08/2026 16:39:19
FILAS: 4
```

Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 6. Inserción de varias filas. Evidencia F14, parte 1/1. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 7 Cursor con productos

Se muestran los dos productos y sus stocks antes de actualizar.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F15 · Cursor con productos · Parte 1/1

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.7. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E08 Alta múltiple ===
FECHA_REAL: 2026-10-08T16:39:19.1243335-03:00
INSERT dbo.Productos(Nombre,Stock,Precio) VALUES(N'Monitor prueba',3,100),
(N'Mouse prueba',5,20); SELECT * FROM dbo.Productos ORDER BY IdProducto;
SELECT * FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
2 | Monitor prueba | 3 | 100.00
3 | Mouse prueba | 5 | 20.00
FILAS: 2
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
3 | 3 | INS | 10/08/2026 16:39:19
4 | 2 | INS | 10/08/2026 16:39:19
FILAS: 4


=== E10 Cursor ===
FECHA_REAL: 2026-10-08T16:39:19.1287763-03:00
EXEC dbo.sp_ConsultarInventarioCursor;
MENSAJE: ID=2; Nombre=Monitor prueba; Stock=3
MENSAJE: ID=3; Nombre=Mouse prueba; Stock=5
```

Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 7. Cursor con productos. Evidencia F15, parte 1/1. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 8 Actualización válida

Los stocks pasan a 4 y 6; se registran dos acciones UPD.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F16 · Actualizacion valida · Parte 1/2

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.8. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E08 Alta múltiple ===
FECHA_REAL: 2026-10-08T16:39:19.1243335-03:00
INSERT dbo.Productos(Nombre,Stock,Precio) VALUES(N'Monitor prueba',3,100),
(N'Mouse prueba',5,20); SELECT * FROM dbo.Productos ORDER BY IdProducto;
SELECT * FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
2 | Monitor prueba | 3 | 100.00
3 | Mouse prueba | 5 | 20.00
FILAS: 2
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
3 | 3 | INS | 10/08/2026 16:39:19
4 | 2 | INS | 10/08/2026 16:39:19
FILAS: 4


=== E10 Actualización válida ===
FECHA_REAL: 2026-10-08T16:39:19.1301546-03:00
UPDATE dbo.Productos SET Stock=Stock+1; SELECT * FROM dbo.Productos ORDER BY
IdProducto; SELECT * FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
2 | Monitor prueba | 4 | 100.00
3 | Mouse prueba | 6 | 20.00
FILAS: 2
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
3 | 3 | INS | 10/08/2026 16:39:19
4 | 2 | INS | 10/08/2026 16:39:19
```

Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 8. Actualización válida. Evidencia F16, parte 1/2. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 8 Actualización válida

Los stocks pasan a 4 y 6; se registran dos acciones UPD.

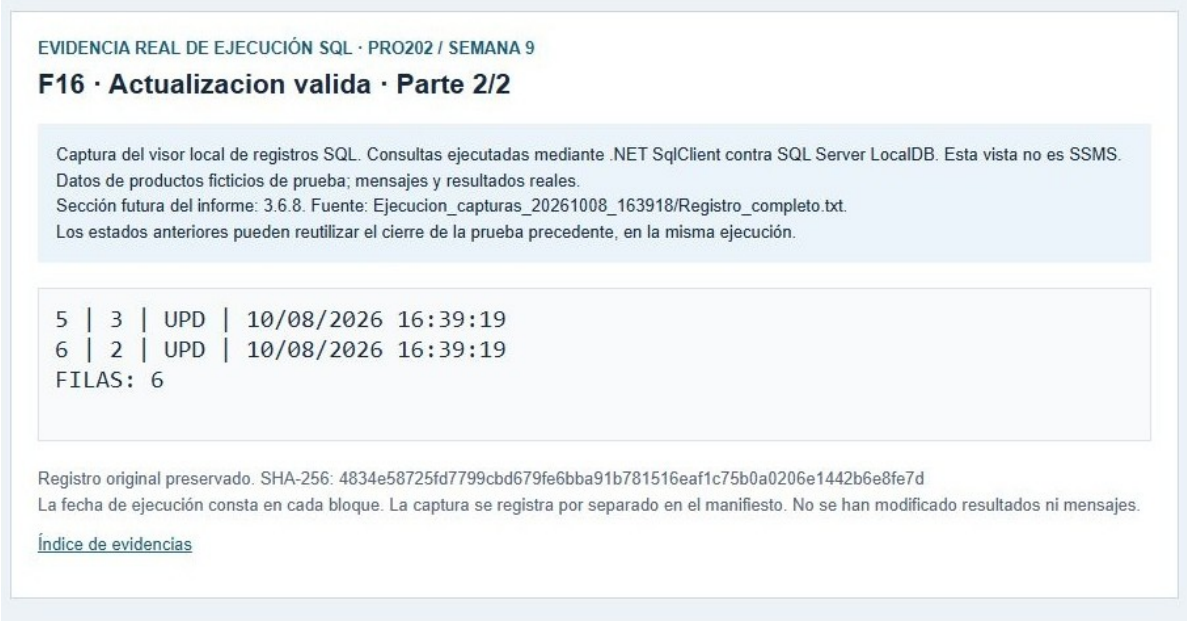


Figura 8. Actualización válida. Evidencia F16, parte 2/2. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 9 Actualización sin cambios

Se mantienen los valores y no se agregan acciones UPD.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F17 · Actualizacion sin cambios · Parte 1/2

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.8. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E10 Actualización válida ===
FECHA_REAL: 2026-10-08T16:39:19.1301546-03:00
UPDATE dbo.Productos SET Stock=Stock+1; SELECT * FROM dbo.Productos ORDER BY
IdProducto; SELECT * FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
2 | Monitor prueba | 4 | 100.00
3 | Mouse prueba | 6 | 20.00
FILAS: 2
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
3 | 3 | INS | 10/08/2026 16:39:19
4 | 2 | INS | 10/08/2026 16:39:19
5 | 3 | UPD | 10/08/2026 16:39:19
6 | 2 | UPD | 10/08/2026 16:39:19
FILAS: 6


=== E10 Actualización sin cambios ===
FECHA_REAL: 2026-10-08T16:39:19.1398629-03:00
UPDATE dbo.Productos SET Stock=Stock; SELECT * FROM dbo.Productos ORDER BY
IdProducto; SELECT * FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
2 | Monitor prueba | 4 | 100.00
3 | Mouse prueba | 6 | 20.00
FILAS: 2
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
```

Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 9. Actualización sin cambios. Evidencia F17, parte 1/2. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 9 Actualización sin cambios

Se mantienen los valores y no se agregan acciones UPD.

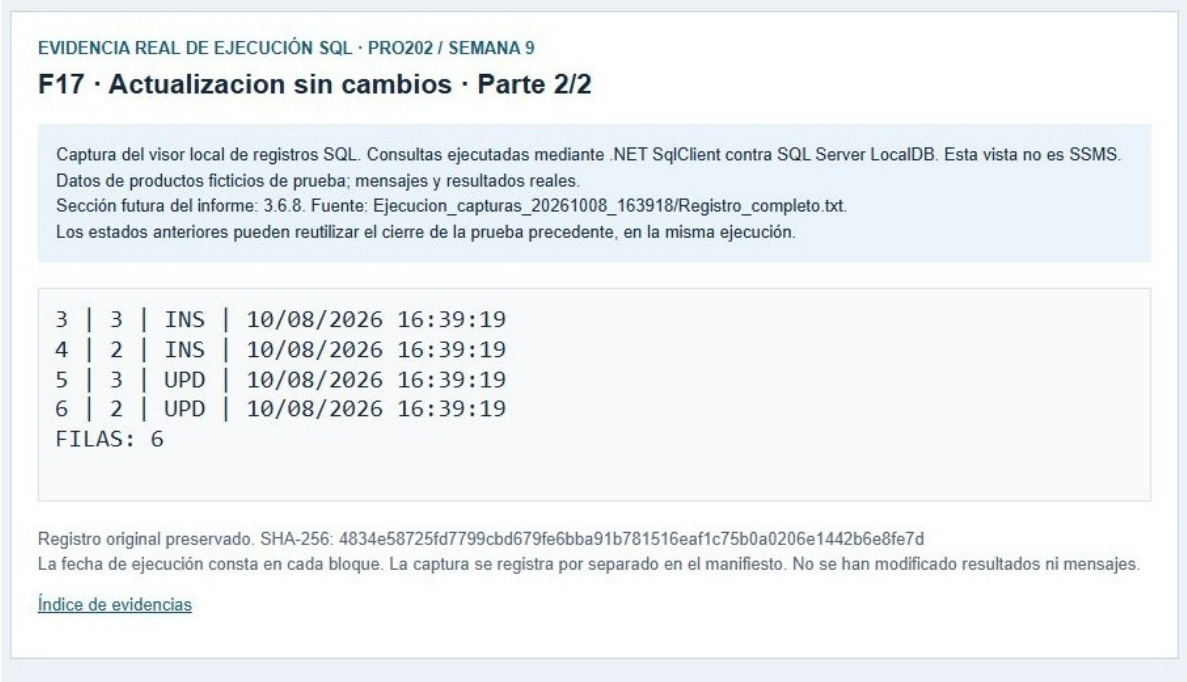


Figura 9. Actualización sin cambios. Evidencia F17, parte 2/2. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 10 Actualización rechazada

El stock negativo genera un error y conserva los datos válidos.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F18 · Actualizacion invalida · Parte 1/2

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.9. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E10 Actualización sin cambios ===
FECHA_REAL: 2026-10-08T16:39:19.1398629-03:00
UPDATE dbo.Productos SET Stock=Stock; SELECT * FROM dbo.Productos ORDER BY
IdProducto; SELECT * FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
2 | Monitor prueba | 4 | 100.00
3 | Mouse prueba | 6 | 20.00
FILAS: 2
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
3 | 3 | INS | 10/08/2026 16:39:19
4 | 2 | INS | 10/08/2026 16:39:19
5 | 3 | UPD | 10/08/2026 16:39:19
6 | 2 | UPD | 10/08/2026 16:39:19
FILAS: 6


=== E09 Actualización inválida ===
FECHA_REAL: 2026-10-08T16:39:19.1427853-03:00
UPDATE dbo.Productos SET Stock=-1;
ERROR: Exception calling "ExecuteReader" with "0" argument(s): "Actualización
rechazada: nombre, stock o precio inválidos."
NUMERO_ERROR_SQL: 50005


=== E09 Estado tras rechazo ===
FECHA_REAL: 2026-10-08T16:39:19.1443490-03:00
SELECT * FROM dbo.Productos ORDER BY IdProducto; SELECT * FROM
dbo.AuditoriaInventario ORDER BY IdAuditoria;
```

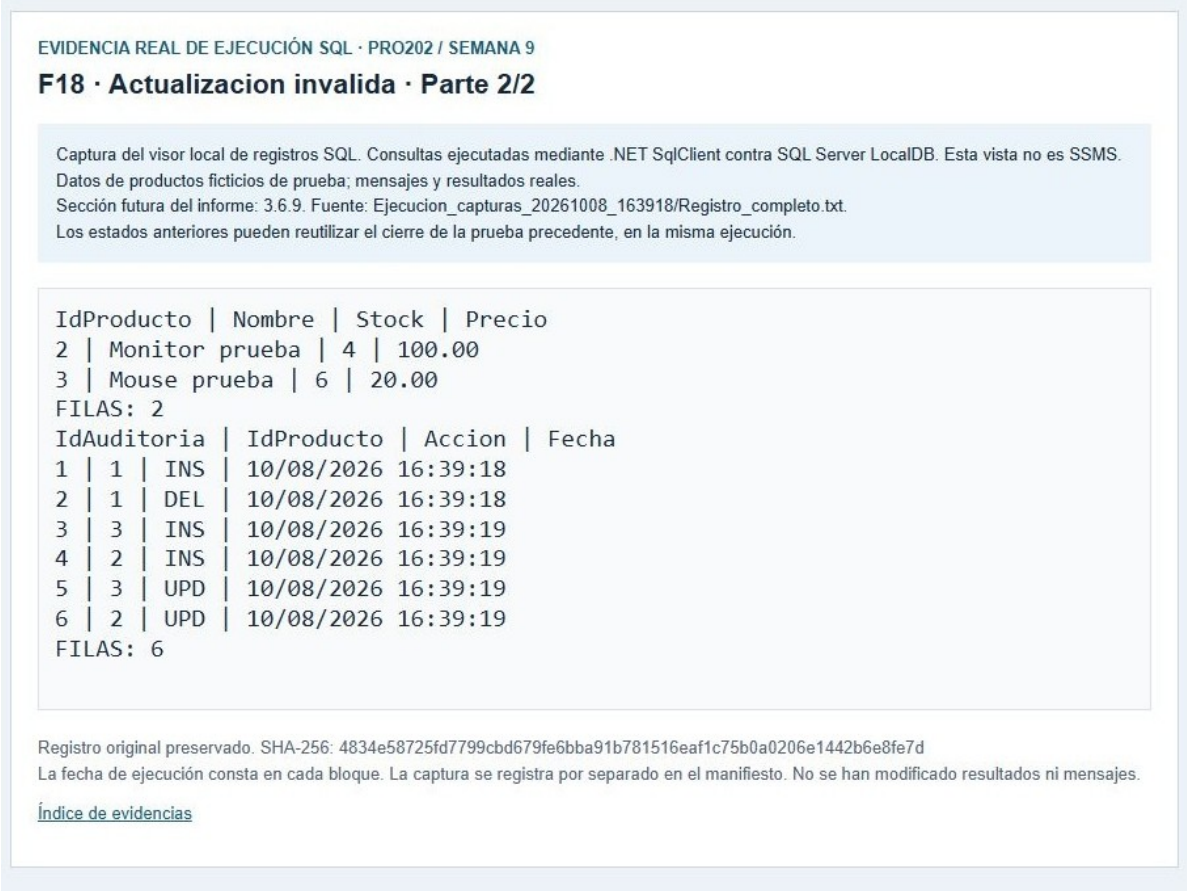
Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 10. Actualización rechazada. Evidencia F18, parte 1/2. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 10 Actualización rechazada

El stock negativo genera un error y conserva los datos válidos.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F18 · Actualizacion invalida · Parte 2/2

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.9. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
IdProducto | Nombre | Stock | Precio
2 | Monitor prueba | 4 | 100.00
3 | Mouse prueba | 6 | 20.00
FILAS: 2
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
3 | 3 | INS | 10/08/2026 16:39:19
4 | 2 | INS | 10/08/2026 16:39:19
5 | 3 | UPD | 10/08/2026 16:39:19
6 | 2 | UPD | 10/08/2026 16:39:19
FILAS: 6
```

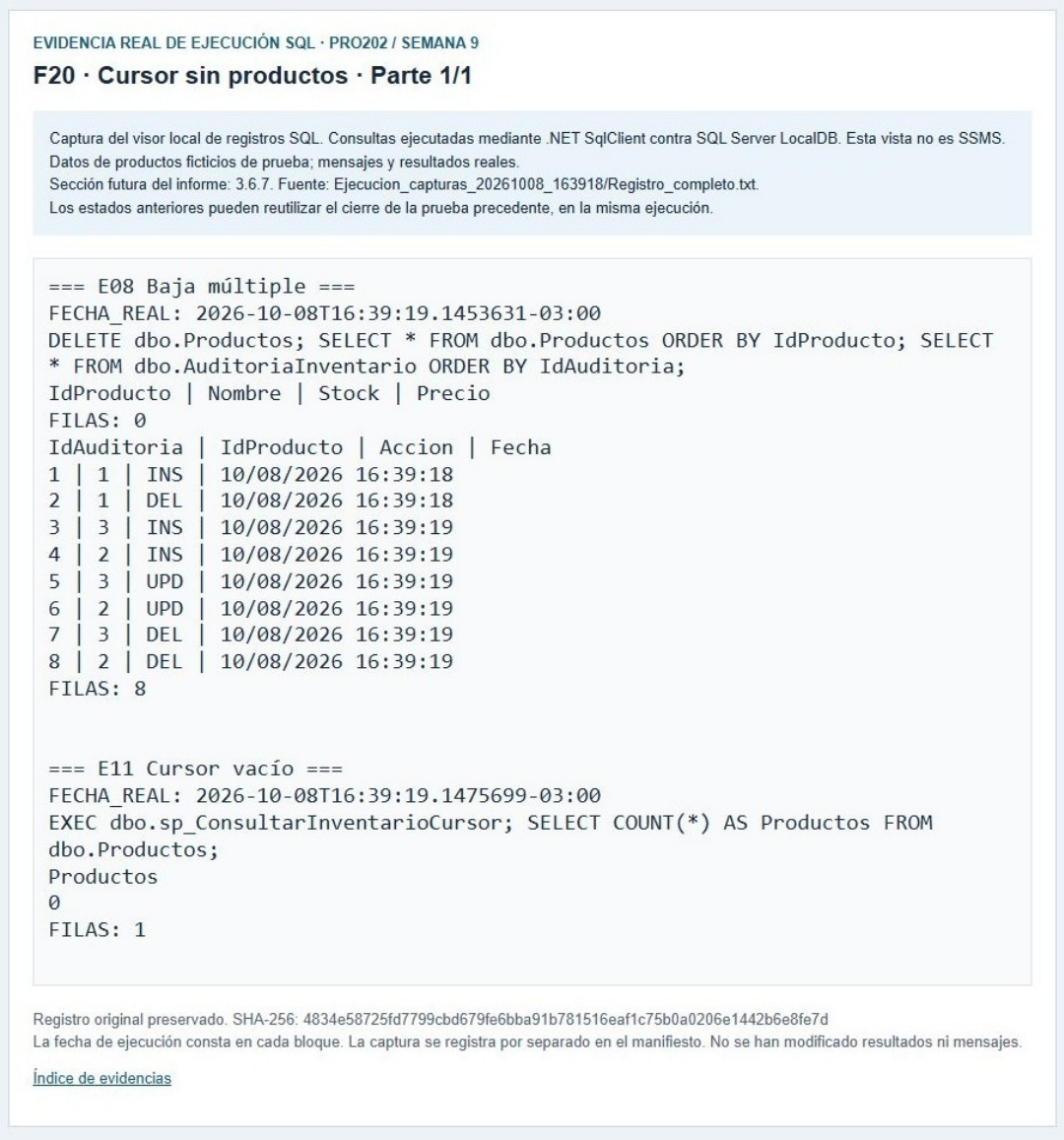
Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 10. Actualización rechazada. Evidencia F18, parte 2/2. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 11 Cursor con inventario vacío

La consulta confirma cero productos; el cursor no emite mensajes de filas.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F20 · Cursor sin productos · Parte 1/1

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.7. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E08 Baja múltiple ===
FECHA_REAL: 2026-10-08T16:39:19.1453631-03:00
DELETE dbo.Productos; SELECT * FROM dbo.Productos ORDER BY IdProducto; SELECT
* FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
FILAS: 0
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
3 | 3 | INS | 10/08/2026 16:39:19
4 | 2 | INS | 10/08/2026 16:39:19
5 | 3 | UPD | 10/08/2026 16:39:19
6 | 2 | UPD | 10/08/2026 16:39:19
7 | 3 | DEL | 10/08/2026 16:39:19
8 | 2 | DEL | 10/08/2026 16:39:19
FILAS: 8


=== E11 Cursor vacío ===
FECHA_REAL: 2026-10-08T16:39:19.1475699-03:00
EXEC dbo.sp_ConsultarInventarioCursor; SELECT COUNT(*) AS Productos FROM
dbo.Productos;
Productos
0
FILAS: 1
```

Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 11. Cursor con inventario vacío. Evidencia F20, parte 1/1. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.

Figura 12 Comprobación final

La validación automática confirma cero productos, cuatro INS, cuatro DEL y dos UPD.



EVIDENCIA REAL DE EJECUCIÓN SQL · PRO202 / SEMANA 9

F35 · Comprobacion final · Parte 1/1

Captura del visor local de registros SQL. Consultas ejecutadas mediante .NET SqlClient contra SQL Server LocalDB. Esta vista no es SSMS.
Datos de productos ficticios de prueba; mensajes y resultados reales.
Sección futura del informe: 3.6.11. Fuente: Ejecucion_capturas_20261008_163918/Registro_completo.txt.
Los estados anteriores pueden reutilizar el cierre de la prueba precedente, en la misma ejecución.

```
=== E14 Limpieza de producto de prueba ===
FECHA_REAL: 2026-10-08T16:39:19.1890322-03:00
DELETE dbo.Productos; SELECT * FROM dbo.Productos ORDER BY IdProducto; SELECT
* FROM dbo.AuditoriaInventario ORDER BY IdAuditoria;
IdProducto | Nombre | Stock | Precio
FILAS: 0
IdAuditoria | IdProducto | Accion | Fecha
1 | 1 | INS | 10/08/2026 16:39:18
2 | 1 | DEL | 10/08/2026 16:39:18
3 | 3 | INS | 10/08/2026 16:39:19
4 | 2 | INS | 10/08/2026 16:39:19
5 | 3 | UPD | 10/08/2026 16:39:19
6 | 2 | UPD | 10/08/2026 16:39:19
7 | 3 | DEL | 10/08/2026 16:39:19
8 | 2 | DEL | 10/08/2026 16:39:19
10 | 5 | INS | 10/08/2026 16:39:19
11 | 5 | DEL | 10/08/2026 16:39:19
FILAS: 10


=== Comprobación final automática ===
FECHA_REAL: 2026-10-08T16:39:19.1903674-03:00
IF (SELECT COUNT(*) FROM dbo.Productos)<>0 THROW 50100,'Productos final
inesperado',1; IF (SELECT COUNT(*) FROM dbo.AuditoriaInventario WHERE
Accion='INS')<>4 THROW 50101,'INS inesperado',1; IF (SELECT COUNT(*) FROM
dbo.AuditoriaInventario WHERE Accion='DEL')<>4 THROW 50102,'DEL inesperado',1;
IF (SELECT COUNT(*) FROM dbo.AuditoriaInventario WHERE Accion='UPD')<>2 THROW
50103,'UPD inesperado',1; SELECT 'Verificaciones superadas' AS Resultado;
Resultado
Verificaciones superadas
FILAS: 1
```

Registro original preservado. SHA-256: 4834e58725fd7799cbd679fe6bba91b781516eaf1c75b0a0206e1442b6e8fe7d
La fecha de ejecución consta en cada bloque. La captura se registra por separado en el manifiesto. No se han modificado resultados ni mensajes.

Índice de evidencias

Figura 12. Comprobación final. Evidencia F35, parte 1/1. Fuente: ejecución SQL del 8 de octubre de 2026, visor local capturado el 9 de octubre de 2026.